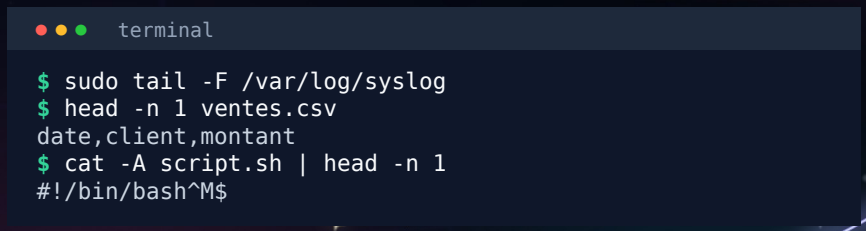
GUIDE PRATIQUE · ADMINISTRATION LINUX

# head, tail et cat sous Linux

## Lire, suivre et assembler des fichiers : de l'usage courant aux astuces d'expert

Début et fin d'un fichier, suivi des journaux en temps réel, assemblage, heredocs, caractères invisibles et combinaisons avec sed, grep et less.




RÉALISÉ PAR

**Ngouye GNING**

VERSION
1.0

DATE
Octobre 2026

CONTENU
21 fiches · 4 parties · annexes

NAVIGATION

# Sommaire

**Introduction** ..... 3

**Les trois commandes en un coup d'œil** ..... 5

**head : lire le début d'un fichier** ..... 7

01 head — Afficher les 10 premières lignes ..... 8
02 head -n — Choisir le nombre de lignes ..... 8
03 head f1 f2 — Plusieurs fichiers à la fois ..... 9
04 head -c — Compter en octets ..... 10
05 head -n -N — Tout sauf les N dernières lignes ..... 10

**tail : lire la fin et suivre les journaux** ..... 11

06 tail — Afficher les 10 dernières lignes ..... 12
07 tail -n — Choisir le nombre de lignes ..... 12
08 tail -f — Suivre un journal en temps réel ..... 13
09 tail -F — Survivre à la rotation des journaux ..... 14
10 tail -n +N — Afficher à partir de la ligne N ..... 15
11 tail -c — Lire les derniers octets ..... 15
12 journalctl -f — L'équivalent pour journald ..... 16

**cat : afficher, assembler, créer** ..... 17

13 cat — Afficher un fichier ..... 18
14 cat f1 f2 > f3 — Assembler des fichiers ..... 19
15 cat > · heredoc — Créer et compléter un fichier ..... 20
16 cat -n · -b — Numéroter les lignes ..... 21
17 cat -A — Révéler les caractères invisibles ..... 22
18 cat | … — Alimenter un pipeline ..... 23

**Aller plus loin : combiner les commandes** ..... 24

19 tac — Lire un fichier à l'envers ..... 25
20 head | tail — Extraire une plage de lignes ..... 26
21 Recettes — Les combinaisons du quotidien ..... 27

**Conclusion** ..... 28

**Annexes** ..... 29

A. Aide-mémoire head ..... 29
B. Aide-mémoire tail ..... 29
C. Aide-mémoire cat et tac ..... 30
D. Où trouver les journaux ? ..... 30
E. Pièges courants ..... 31
F. Sources et références ..... 31
G. Glossaire ..... 32

AVANT-PROPOS

# Introduction

Chaque système Linux — machine virtuelle dans le cloud, serveur physique ou conteneur — produit en permanence des fichiers texte : configurations, journaux, exports CSV, comptes rendus de déploiement, traces de tâches cron. Savoir les lire et les examiner en quelques secondes, sans ouvrir d'éditeur ni écrire de script, est l'une des compétences les plus rentables de la ligne de commande.




Pourtant, la plupart des administrateurs n'utilisent `cat` que pour afficher un fichier, et `tail -f` que pour suivre un journal. Les trois commandes de ce guide — `head`, `tail` et `cat` — existent depuis les débuts d'Unix et sont présentes sur toutes les distributions, mais une bonne partie de leurs possibilités reste méconnue :

- `head` sait afficher **tout sauf les N dernières lignes** d'un fichier, et pas seulement les premières ;
- `tail` peut suivre un journal **par son nom**, et survivre ainsi à la rotation des journaux ;
- `cat` révèle les **caractères invisibles** — tabulations, fins de ligne Windows — qui font échouer des scripts sans raison apparente.

## Objectifs du guide

- Lire rapidement le **début**, la **fin** ou une **portion précise** d'un fichier ;
- **Surveiller des journaux** en temps réel, y compris pendant une rotation et avec journald ;
- **Créer, assembler et diagnostiquer** des fichiers avec `cat`, les redirections et les heredocs ;
- **Combiner** ces commandes avec les tubes et les autres outils texte (`grep`, `sed`, `awk`, `less`).

## Prérequis et environnement

Une connaissance de base du terminal et de la navigation dans l'arborescence suffit. Les exemples sont présentés pour Ubuntu et Debian ; les différences avec RHEL, Rocky Linux et AlmaLinux — notamment l'emplacement des journaux — sont signalées. Les droits root ne sont nécessaires que pour lire certains journaux système : `sudo` est alors indiqué.

## Conventions utilisées

| NOTATION | SIGNIFICATION |
|---|---|
| `$ commande` | Commande exécutée par un utilisateur standard |
| `$ sudo commande` | Commande nécessitant les privilèges administrateur (root) |
| `fichier`, `N` | Nom de fichier ou nombre à remplacer par votre propre valeur |
| `# texte` | Commentaire explicatif dans un bloc de code |
| Essentiel Pratique Avancé | Niveau de chaque fiche : à connaître par cœur, d'usage courant, astuce d'expert |

> **À PROPOS DES EXEMPLES**
> Les sorties présentées sont illustratives (machine `srv01`). Selon la configuration de rsyslog, l'horodatage des journaux peut apparaître au format classique (`Aug 24 15:55:01`) ou au format ISO 8601 (`2026-08-24T15:55:01.529373+02:00`) ; les commandes, elles, fonctionnent à l'identique.

SYNTHÈSE

# Les trois commandes en un coup d'œil

`head` lit le début d'un fichier, `tail` sa fin — et le suit en temps réel —, `cat` l'affiche en entier ou l'assemble avec d'autres. Combinées entre elles, ou avec `tac` et `sed`, elles extraient n'importe quelle portion d'un fichier.

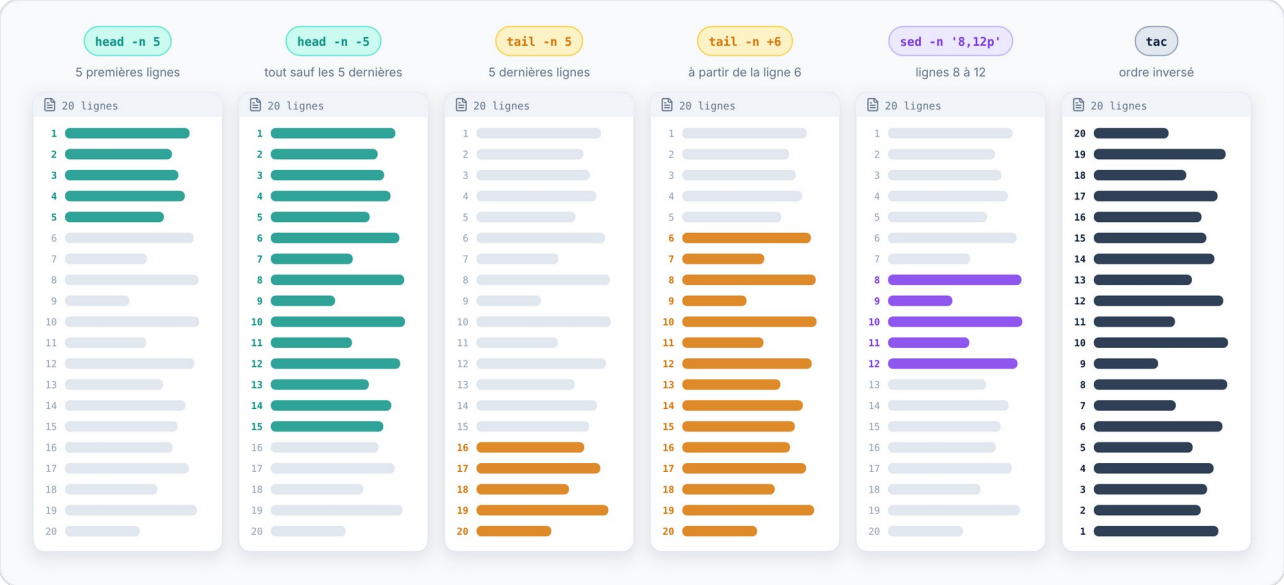



**Figure 1** — *Sur un fichier de 20 lignes : la portion affichée par chaque commande.*

## Référence rapide

| COMMANDE | RÉSULTAT | NIVEAU |
|---|---|---|
| `head fichier` | Les 10 premières lignes | Essentiel |
| `head -n 20 fichier` | Les 20 premières lignes | Essentiel |
| `head -n -5 fichier` | Tout sauf les 5 dernières lignes | Avancé |
| `head -c 100 fichier` | Les 100 premiers octets | Pratique |
| `tail fichier` | Les 10 dernières lignes | Essentiel |
| `tail -n +2 fichier` | De la ligne 2 jusqu'à la fin | Pratique |
| `tail -f fichier` | Suivre les ajouts en temps réel | Essentiel |
| `tail -F fichier` | Suivre par nom, malgré la rotation des journaux | Avancé |
| `journalctl -u service -f` | Suivre le journal d'un service systemd | Pratique |
| `cat fichier` | Afficher tout le fichier | Essentiel |
| `cat f1 f2 > f3` | Concaténer dans un nouveau fichier | Essentiel |
| `cat -n fichier` | Numéroter les lignes | Pratique |
| `cat -A fichier` | Révéler tabulations et fins de ligne | Avancé |
| `tac fichier` | Afficher les lignes en ordre inverse | Pratique |
| `sed -n '20,30p' fichier` | Afficher les lignes 20 à 30 | Pratique |

Essentiel à connaître par cœur Pratique d'usage courant Avancé astuce d'expert

> **À RETENIR**
> Avant d'ouvrir un gros fichier dans un éditeur, regardez-en d'abord le début avec `head` ou la fin avec `tail` : c'est instantané, même sur un journal de plusieurs gigaoctets, et sans aucun risque de modification accidentelle.




# head : lire le début d'un fichier

`head` affiche le début d'un fichier — par défaut ses 10 premières lignes. C'est le moyen le plus rapide de vérifier un format, un en-tête de colonnes ou les premières entrées d'un fichier, sans ouvrir d'éditeur.




## DANS CETTE PARTIE


| | | |
|---|---|---|
| 01 | `head` | Afficher les 10 premières lignes |
| 02 | `head -n` | Choisir le nombre de lignes |
| 03 | `head f1 f2` | Plusieurs fichiers à la fois |
| 04 | `head -c` | Compter en octets |
| 05 | `head -n -N` | Tout sauf les N dernières lignes |

FICHE 01 · HEAD

# 01 head — Afficher les 10 premières lignes

| OBJECTIF | DISPONIBILITÉ | NIVEAU |
|---|---|---|
| Jeter un œil au début d'un fichier | Toutes distributions (GNU coreutils) | Essentiel |

Syntaxe : `head [options] [fichier…]`. Sans option, `head` affiche les 10 premières lignes de chaque fichier indiqué :

bash

```
$ head /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
```

Chaque ligne de `/etc/passwd` décrit un compte : nom, mot de passe (stocké ailleurs, d'où le `x`), UID, GID, description, répertoire personnel et shell de connexion.

FICHE 02 · HEAD

# 02 head -n — Choisir le nombre de lignes

| OBJECTIF | DISPONIBILITÉ | NIVEAU |
|---|---|---|
| Afficher exactement N lignes | Toutes distributions (GNU coreutils) | Essentiel |

L'option `-n` fixe le nombre de lignes affichées. La forme abrégée `-5` est équivalente et très répandue :

bash

```
$ sudo head -n 5 /var/log/syslog
$ sudo head -5 /var/log/syslog          # forme abrégée équivalente
Aug 18 05:06:36 srv01 systemd[1]: Starting logrotate.service - Rotate log files...
Aug 18 05:06:36 srv01 rsyslogd: rsyslogd was HUPed
Aug 18 05:06:36 srv01 systemd[1]: logrotate.service: Deactivated successfully.
Aug 18 05:06:36 srv01 systemd[1]: Finished logrotate.service - Rotate log files.
Aug 18 05:09:01 srv01 CRON[2214]: (root) CMD (/usr/local/bin/backup.sh)
```

> **ASTUCE**
> Sur RHEL, Rocky Linux et AlmaLinux, le journal système principal s'appelle `/var/log/messages`. Sur Debian 12 et les conteneurs minimaux, `/var/log/syslog` peut ne pas exister : les journaux sont alors dans journald (fiche 12).

FICHE 03 · HEAD

# 03 head f1 f2 — Plusieurs fichiers à la fois

| OBJECTIF | DISPONIBILITÉ | NIVEAU |
| --- | --- | --- |
| Comparer le début de plusieurs fichiers | Toutes distributions (GNU coreutils) | Pratique |

`head` accepte plusieurs fichiers : chaque extrait est alors précédé d'un en-tête `==> nom <==` qui indique sa provenance.

bash

```
$ head -n 3 /etc/passwd /etc/group
==> /etc/passwd <==
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin

==> /etc/group <==
root:x:0:
daemon:x:1:
bin:x:2:
```

| OPTION | EFFET |
| --- | --- |
| `-q` | Ne jamais afficher les en-têtes (--quiet) |
| `-v` | Toujours afficher l'en-tête, même pour un seul fichier (--verbose) |

CAS PRATIQUE

`head -n 1 *.csv` affiche d'un coup la ligne d'en-tête de tous les fichiers CSV d'un dossier : idéal pour vérifier que leurs colonnes concordent avant de les assembler.

FICHE 04 · HEAD

## 04 head -c — Compter en octets

| OBJECTIF | DISPONIBILITÉ | NIVEAU |
|---|---|---|
| Lire les premiers octets d'un fichier | Toutes distributions (GNU coreutils) | Pratique |

Avec `-c`, `head` compte en octets plutôt qu'en lignes. Utile pour inspecter le début d'un fichier très volumineux ou binaire, ou pour vérifier sa signature :

bash

```
$ sudo head -c 50 /var/log/syslog
Aug 18 05:06:36 srv01 systemd[1]: Starting logrota
# Identifier un fichier par ses premiers octets (signature PNG)
$ head -c 4 image.png | od -c
0000000 211   P   N   G
0000004
```

**BON À SAVOIR**

Les suffixes `K`, `M` et `G` sont acceptés (`head -c 1K`). Couper un texte au milieu d'un caractère accentué — codé sur plusieurs octets en UTF-8 — peut afficher un symbole illisible en fin de sortie : c'est normal avec `-c`.

FICHE 05 · HEAD

## 05 head -n -N — Tout sauf les N dernières lignes

| OBJECTIF | DISPONIBILITÉ | NIVEAU |
|---|---|---|
| Retirer la fin d'un fichier à l'affichage | GNU coreutils (Linux) · absent de macOS/BSD | Avancé |

Fonction propre à la version GNU : un nombre **négatif** affiche tout le fichier **sauf** les N dernières lignes. Pratique pour retirer une ligne de total ou de pied de page d'un export, ou ignorer les dernières entrées d'un journal :

bash

```
# Tout sauf les 5 dernières lignes
$ head -n -5 access.log
# Retirer la ligne de total d'un export CSV
$ head -n -1 export.csv > export_sans_total.csv
# Même principe en octets : tout sauf les 100 derniers
$ head -c -100 fichier.txt
```

**PORTABILITÉ**

Sur macOS et les BSD, `head` n'accepte pas de valeur négative. Installez les GNU coreutils (commande `ghead`) ou, pour retirer la seule dernière ligne, utilisez `sed '$d' fichier`.




# tail : lire la fin et suivre les journaux

`tail` affiche la fin d'un fichier — par défaut ses 10 dernières lignes — et sait surtout le suivre en temps réel. C'est l'outil de base de tout dépannage : on relance un service, on observe son journal, on comprend.




## DANS CETTE PARTIE


| | | |
|---|---|---|
| 06 | `tail` | Afficher les 10 dernières lignes |
| 07 | `tail -n` | Choisir le nombre de lignes |
| 08 | `tail -f` | Suivre un journal en temps réel |
| 09 | `tail -F` | Survivre à la rotation des journaux |
| 10 | `tail -n +N` | Afficher à partir de la ligne N |
| 11 | `tail -c` | Lire les derniers octets |
| 12 | `journalctl -f` | L'équivalent pour journald |

FICHE 06 · TAIL

## 06 `tail` — Afficher les 10 dernières lignes

| OBJECTIF | DISPONIBILITÉ | NIVEAU |
| --- | --- | --- |
| Voir les événements les plus récents | Toutes distributions (GNU coreutils) | Essentiel |

Syntaxe : `tail [options] [fichier…]`. Sans option, `tail` affiche les 10 dernières lignes, c'est-à-dire, dans un journal, les événements les plus récents :

bash

```
$ sudo tail /var/log/syslog
# … (extrait des 10 lignes affichées)
Aug 24 15:55:01 srv01 CRON[17986]: (admin) CMD (/usr/local/bin/backup.sh)
Aug 24 15:56:15 srv01 systemd[1]: man-db.service: Deactivated successfully.
Aug 24 15:56:15 srv01 systemd[1]: Finished man-db.service - Daily man-db regeneration.
Aug 24 16:00:01 srv01 CRON[18307]: (admin) CMD (/usr/local/bin/backup.sh)
```

FICHE 07 · TAIL

## 07 `tail -n` — Choisir le nombre de lignes

| OBJECTIF | DISPONIBILITÉ | NIVEAU |
| --- | --- | --- |
| Afficher exactement les N dernières lignes | Toutes distributions (GNU coreutils) | Essentiel |

Comme pour `head`, l'option `-n` fixe le nombre de lignes, et la forme abrégée `-20` est équivalente. Exemple avec le journal d'authentification, qui trace les connexions SSH et l'usage de `sudo` :

bash

```
$ sudo tail -n 20 /var/log/auth.log
$ sudo tail -20 /var/log/auth.log       # forme abrégée équivalente
# RHEL / Rocky / AlmaLinux : ce journal s'appelle /var/log/secure
$ sudo tail -n 20 /var/log/secure
```

FICHE 08 · TAIL

# 08 tail -f — Suivre un journal en temps réel

| OBJECTIF | DISPONIBILITÉ | NIVEAU |
|---|---|---|
| Observer un service pendant un dépannage | Toutes distributions (GNU coreutils) | Essentiel |

L'option `-f` (*follow*) garde le fichier ouvert et affiche chaque nouvelle ligne dès qu'elle est écrite. C'est la méthode classique pour observer un service pendant qu'on le teste ; **Ctrl + C** arrête le suivi.

bash

```
$ sudo tail -f /var/log/nginx/error.log
# Afficher d'abord les 50 dernières lignes, puis suivre
$ sudo tail -n 50 -f /var/log/nginx/access.log
# Suivre plusieurs journaux à la fois : un en-tête signale chaque changement
$ sudo tail -f /var/log/nginx/access.log /var/log/nginx/error.log
==> /var/log/nginx/access.log <==
192.168.1.10 - - [24/Aug/2026:16:02:11 +0200] "GET / HTTP/1.1" 200 615 "-" "curl/8.5.0"
```

| OPTION | EFFET |
|---|---|
| `-f` | Suivre le fichier ouvert (son descripteur) |
| `-n N -f` | Afficher les N dernières lignes, puis suivre |
| `-s N` | Vérifier les ajouts toutes les N secondes |
| `--pid=PID` | Arrêter le suivi quand le processus PID se termine |
| `-q` | Ne pas afficher les en-têtes de fichiers |

**FILTRER EN DIRECT**

Pour ne garder que les lignes utiles pendant le suivi, ajoutez `grep` : `sudo tail -f /var/log/nginx/error.log | grep --line-buffered -i 'error'`. L'option `--line-buffered` évite que `grep` ne retienne ses résultats lorsque la sortie est elle-même redirigée.

FICHE 09 · TAIL

# 09 tail -F — Survivre à la rotation des journaux

| OBJECTIF | DISPONIBILITÉ | NIVEAU |
| --- | --- | --- |
| Surveiller un journal sur la durée | Toutes distributions (GNU coreutils) | Avancé |

Les journaux sont régulièrement « tournés » par logrotate : le fichier courant est renommé (`syslog` devient `syslog.1`) puis un nouveau fichier vide est créé. Or `tail -f` suit le **descripteur** du fichier ouvert au départ : après la rotation, il continue de surveiller l'ancien fichier, qui ne reçoit plus rien.

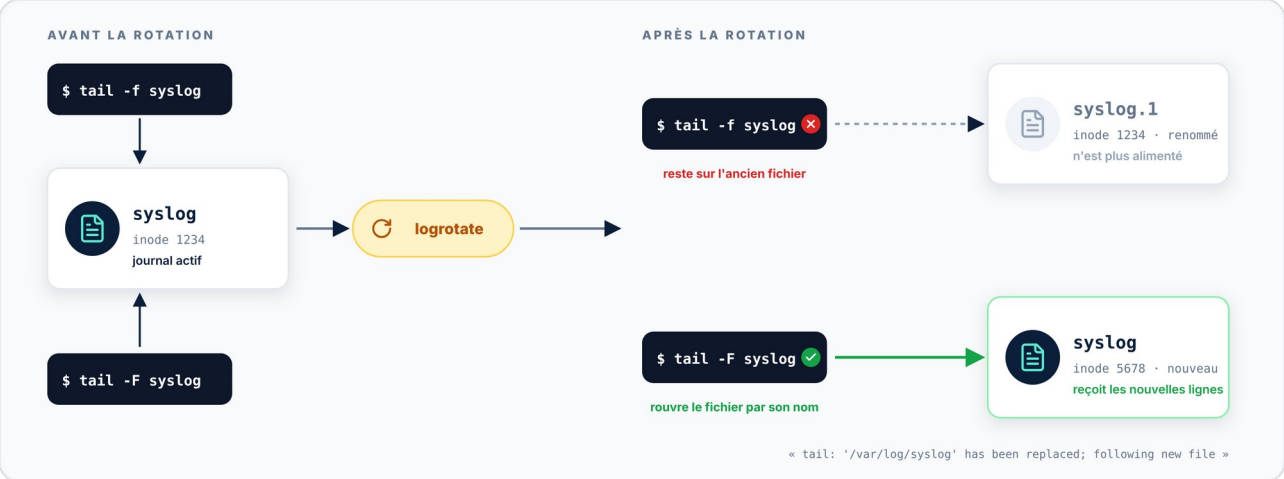



**Figure 2** — *Pendant une rotation, tail -f reste sur l'ancien fichier ; tail -F rouvre le nouveau.*

Avec `--follow=name`, `tail` suit le fichier **par son nom** et le rouvre lorsqu'il est remplacé. L'option `-F` combine `--follow=name` et `--retry` (réessayer si le fichier est momentanément absent) : c'est la forme à retenir.

```bash
$ sudo tail --follow=name /var/log/syslog
# Forme courte recommandée (= --follow=name --retry)
$ sudo tail -F /var/log/syslog
# Message affiché au moment d'une rotation
tail: '/var/log/syslog' has been replaced;  following new file
# RHEL / Rocky / AlmaLinux
$ sudo tail -F /var/log/messages
```

> **LA BONNE HABITUDE**
> Pour toute surveillance longue — une nuit, un week-end, une campagne de tests —, utilisez `tail -F` plutôt que `tail -f` : c'est la seule forme qui survit aux rotations.

FICHE 10 · TAIL

## 10 `tail -n +N` — Afficher à partir de la ligne N

| OBJECTIF | DISPONIBILITÉ | NIVEAU |
| --- | --- | --- |
| Ignorer un en-tête ou les premières lignes | Toutes distributions (GNU coreutils) | Pratique |

Précédé d'un `+`, le nombre désigne la ligne **de départ** au lieu d'un nombre de lignes comptées depuis la fin. `tail -n +2` affiche donc tout le fichier à partir de la ligne 2 — idéal pour ignorer une ligne d'en-tête :

bash

```
# Ignorer la première ligne
$ tail -n +2 /etc/passwd
# Retirer l'en-tête d'un CSV, puis trier sur la 3e colonne
$ tail -n +2 ventes.csv | sort -t, -k3 -n
# Compter les enregistrements d'un CSV, en-tête exclu
$ tail -n +2 ventes.csv | wc -l
```

FICHE 11 · TAIL

## 11 `tail -c` — Lire les derniers octets

| OBJECTIF | DISPONIBILITÉ | NIVEAU |
| --- | --- | --- |
| Examiner la toute fin d'un fichier | Toutes distributions (GNU coreutils) | Pratique |

Comme `head`, `tail` peut compter en octets avec `-c`. Utile pour lire la fin d'un fichier sans retours à la ligne, ou vérifier comment se termine un fichier :

bash

```
$ sudo tail -c 100 /var/log/syslog
# Le fichier se termine-t-il par un saut de ligne ?
$ tail -c 1 script.sh | od -c
0000000  \n
0000001
```

**POURQUOI C'EST UTILE**

Un fichier texte doit se terminer par un saut de ligne. Sans lui, `cat` colle l'invite du shell à la dernière ligne et certains outils ignorent purement et simplement cette ligne.

FICHE 12 · TAIL

# 12 journalctl -f — L'équivalent pour journald

| OBJECTIF | DISPONIBILITÉ | NIVEAU |
| --- | --- | --- |
| Suivre les services qui écrivent dans journald | Systèmes systemd (Ubuntu, Debian, RHEL, Rocky…) | Pratique |

Sur les systèmes récents, de nombreux services écrivent dans le journal de systemd (journald) plutôt que dans des fichiers texte. `journalctl` offre alors exactement les mêmes usages que `tail` :

| AVEC TAIL (FICHIER TEXTE) | AVEC JOURNALCTL (JOURNALD) |
| --- | --- |
| `tail -n 50 fichier` | `journalctl -n 50` |
| `tail -f fichier` | `journalctl -f` |
| `tail -n 50 -f fichier` | `journalctl -n 50 -f` |
| `tail -f /var/log/nginx/error.log` | `journalctl -u nginx -f` |

bash

```
# Suivre le journal du service nginx
$ sudo journalctl -u nginx -f
# 50 dernières entrées de SSH, puis suivi (ssh sur Debian/Ubuntu, sshd sur RHEL)
$ sudo journalctl -u ssh -n 50 -f
# Uniquement les erreurs des 10 dernières minutes
$ sudo journalctl -p err --since '10 min ago'
```

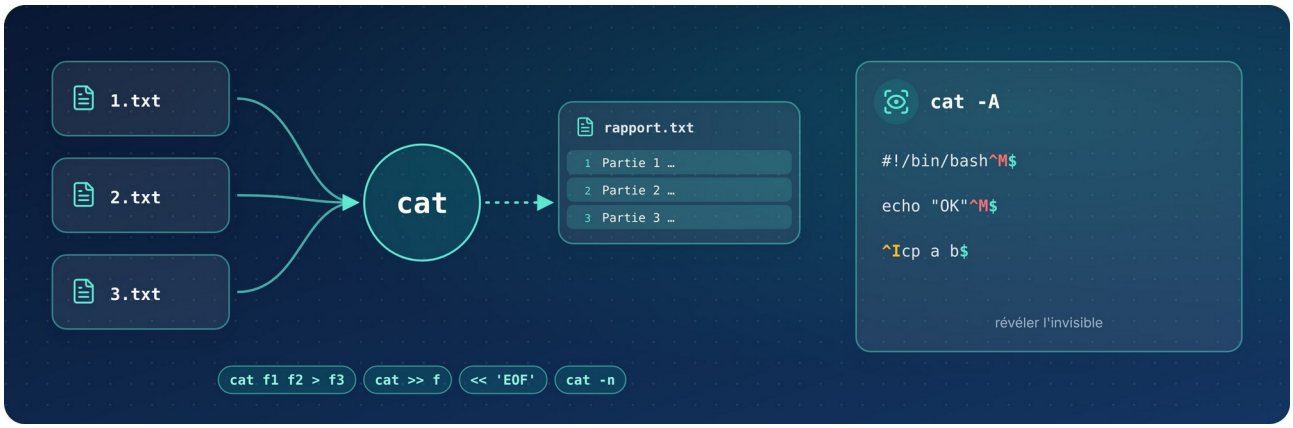
**FICHIERS OU JOURNALD ?**

Si `/var/log/<service>.log` n'existe pas ou reste vide, le service écrit probablement dans journald : `systemctl status <service>` affiche d'ailleurs ses dernières lignes de journal.




# cat : afficher, assembler, créer

`cat` (pour *concatenate*) lit un ou plusieurs fichiers et écrit leur contenu sur la sortie standard. Associé aux redirections, il sert aussi à créer, compléter et assembler des fichiers — et ses options révèlent ce qu'un éditeur ne montre pas.




## DANS CETTE PARTIE


| | | |
|---|---|---|
| 13 | `cat` | Afficher un fichier |
| 14 | `cat f1 f2 > f3` | Assembler des fichiers |
| 15 | `cat > · heredoc` | Créer et compléter un fichier |
| 16 | `cat -n · -b` | Numéroter les lignes |
| 17 | `cat -A` | Révéler les caractères invisibles |
| 18 | `cat \| …` | Alimenter un pipeline |

FICHE 13 · CAT

# 13 cat — Afficher un fichier

| OBJECTIF | DISPONIBILITÉ | NIVEAU |
| --- | --- | --- |
| Lire un fichier court en entier | Toutes distributions (GNU coreutils) | Essentiel |

Syntaxe : `cat [options] [fichier…]`. Le cas le plus simple : afficher un fichier court dans le terminal, par exemple pour identifier la distribution installée.

bash

```
$ cat /etc/os-release
PRETTY_NAME="Ubuntu 24.04.3 LTS"
NAME="Ubuntu"
VERSION_ID="24.04"
VERSION="24.04.3 LTS (Noble Numbat)"
VERSION_CODENAME=noble
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
UBUNTU_CODENAME=noble
# Plusieurs fichiers sont affichés l'un après l'autre, sans séparation
$ cat /etc/hostname /etc/hosts
```

**FICHIERS LONGS OU BINAIRES**

Pour un fichier de plusieurs centaines de lignes, préférez `less`, qui permet de défiler et de rechercher. Et n'affichez jamais un fichier binaire avec `cat` : il peut dérégler le terminal (la commande `reset` le rétablit).

FICHE 14 · CAT

# 14 cat f1 f2 > f3 — Assembler des fichiers

| OBJECTIF | DISPONIBILITÉ | NIVEAU |
|---|---|---|
| Réunir plusieurs fichiers en un seul | Toutes distributions (GNU coreutils) | Essentiel |

La sortie de `cat` peut être redirigée : vers un fichier avec `>` (qui le remplace) ou `>>` (qui ajoute à la fin), ou vers une autre commande avec un tube `|`. C'est ainsi qu'on assemble plusieurs fichiers en un seul, dans l'ordre indiqué.

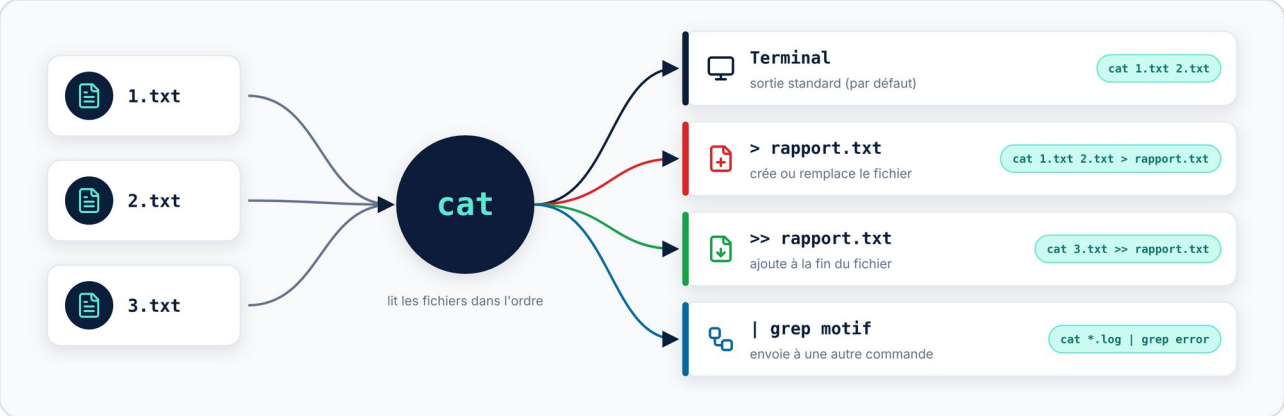



**Figure 3** — *cat lit ses fichiers d'entrée dans l'ordre ; la redirection décide où va le résultat.*

```bash
$ echo 'Partie 1 : introduction' > 1.txt
$ echo 'Partie 2 : analyse' > 2.txt
$ echo 'Partie 3 : conclusion' > 3.txt
$ cat 1.txt 2.txt 3.txt > rapport.txt
$ cat rapport.txt
Partie 1 : introduction
Partie 2 : analyse
Partie 3 : conclusion
# Cas concrets : fragments de configuration, segments de journal
$ cat header.txt body.txt footer.txt > report.txt
$ cat access.log.2 access.log.1 access.log > access-complet.log
```

> **> REMPLACE, >> AJOUTE**
> `>` écrase le fichier de destination sans avertissement. Piège classique : `cat a.txt b.txt > a.txt` **vide** `a.txt` avant même que `cat` ne le lise, car le shell prépare la redirection en premier. Passez par un fichier temporaire. Pour une simple copie, `cp` reste préférable à `cat source > copie`.

FICHE 15 · CAT

# 15 cat > · heredoc — Créer et compléter un fichier

| OBJECTIF | DISPONIBILITÉ | NIVEAU |
|---|---|---|
| Écrire un fichier depuis le terminal | Toutes distributions (GNU coreutils) | Pratique |

Sans fichier d'entrée, `cat` lit le clavier : après `cat > notes.txt`, tout ce que vous tapez est écrit dans le fichier, ligne après ligne. **Ctrl + D** (fin de fichier) termine la saisie et rend la main.

## Le heredoc : créer un fichier en une seule commande

Pour des créations répétables — scripts, déploiements, documentation —, le *heredoc* est plus pratique : tout le contenu figure dans la commande, entre deux marqueurs identiques.

bash

```
$ cat > /tmp/app.conf << 'EOF'
server_name=web01
env=production
port=8080
EOF
$ cat /tmp/app.conf
server_name=web01
env=production
port=8080
```

| FORME | EFFET |
|---|---|
| `<< 'EOF'` | Contenu copié tel quel : `$VAR` et `$(commande)` ne sont **pas** interprétés |
| `<< EOF` | Les variables et substitutions de commandes sont remplacées par leur valeur |
| `<<- EOF` | Les tabulations de début de ligne sont ignorées (indentation dans un script) |
| `<< FIN` | N'importe quel mot convient, s'il est identique à l'ouverture et à la fermeture |

## Ajouter à la fin d'un fichier

bash

```
$ cat >> ~/deploy.log << 'EOF'
Déploiement terminé le 24/08/2026 à 14:32 UTC
EOF
# Fichier appartenant à root : passer par sudo tee -a
$ sudo tee -a /var/log/deploy.log > /dev/null << 'EOF'
Déploiement terminé le 24/08/2026 à 14:32 UTC
EOF
```

SUDO ET REDIRECTION

`sudo cat >> /var/log/deploy.log` échoue avec « Permission denied » : la redirection est réalisée par votre propre shell, sans privilèges. `sudo tee -a` écrit, lui, le fichier avec les droits root.

FICHE 16 · CAT

# 16 cat -n · -b — Numéroter les lignes

| OBJECTIF | DISPONIBILITÉ | NIVEAU |
| --- | --- | --- |
| Repérer et citer une ligne précise | Toutes distributions (GNU coreutils) | Pratique |

`-n` numérote toutes les lignes, y compris les lignes vides ; `-b` ne numérote que les lignes non vides. Pratique pour situer une ligne citée dans un message d'erreur ou pour échanger avec un collègue.

bash

```
$ cat -n /etc/hosts
     1  127.0.0.1 localhost
     2  127.0.1.1 srv01
     3
     4  # The following lines are desirable for IPv6 capable hosts
     5  ::1     ip6-localhost ip6-loopback
# Numéroter seulement les lignes non vides
$ cat -b /etc/nginx/nginx.conf
```

| OPTION | EFFET |
| --- | --- |
| `-n` | Numéroter toutes les lignes |
| `-b` | Numéroter uniquement les lignes non vides (prioritaire sur -n) |
| `-s` | Fusionner les lignes vides consécutives en une seule |

FICHE 17 · CAT

# 17 cat -A — Révéler les caractères invisibles

| OBJECTIF | DISPONIBILITÉ | NIVEAU |
|---|---|---|
| Diagnostiquer un script ou une configuration | Toutes distributions (GNU coreutils) | Avancé |

C'est l'option la plus précieuse en dépannage. `-A` rend visible ce qu'aucun éditeur ne montre : les tabulations apparaissent sous la forme `^I`, la fin de chaque ligne est marquée par `$` et le retour chariot des fichiers Windows (CRLF) apparaît en `^M`.

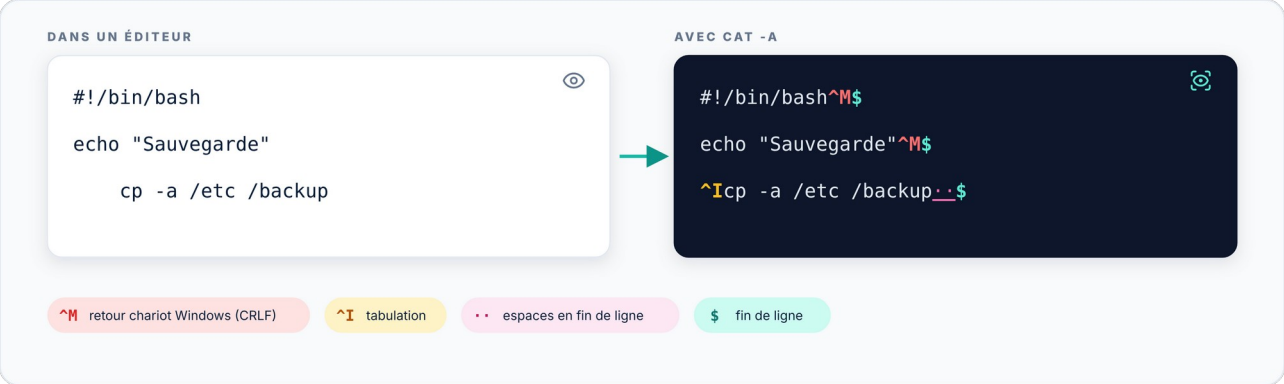



Figure 4 — *Ce que montre un éditeur… et ce que révèle cat -A.*

```bash
$ ./suspect.sh
-bash: ./suspect.sh: /bin/bash^M: bad interpreter: No such file or directory
$ cat -A suspect.sh
#!/bin/bash^M$
echo "Sauvegarde"^M$
^Icp -a /etc /backup  $
# Corriger : supprimer les retours chariot Windows
$ sed -i 's/\r$//' suspect.sh          # ou : dos2unix suspect.sh
```

| OPTION | EFFET |
|---|---|
| `-A` | Tout révéler (équivaut à -vET) |
| `-E` | Afficher `$` en fin de ligne : repère les espaces finaux |
| `-T` | Afficher les tabulations sous la forme ^I |
| `-v` | Afficher les autres caractères non imprimables (^M…) |

FICHE 18 · CAT

# 18 cat | … — Alimenter un pipeline

| OBJECTIF | DISPONIBILITÉ | NIVEAU |
| --- | --- | --- |
| Envoyer un fichier à d'autres outils | Toutes distributions (GNU coreutils) | Pratique |

`cat` alimente volontiers d'autres commandes via un tube (`|`). Exemple : classer les adresses IP à l'origine d'échecs de connexion SSH, de la plus fréquente à la moins fréquente :

bash

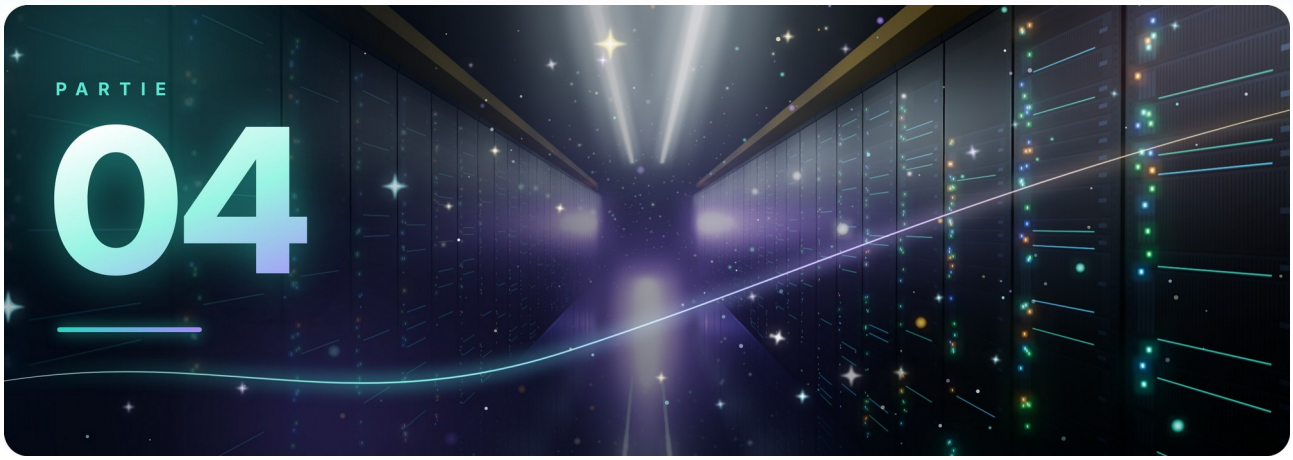
```
$ sudo cat /var/log/auth.log | grep 'Failed password' \
    | grep -oE 'from [0-9.]+' | awk '{print $2}' | sort | uniq -c | sort -rn
     42 203.0.113.7
     17 198.51.100.23
      3 192.168.1.50
```

Extraire l'adresse avec `grep -oE 'from [0-9.]+'` plutôt qu'avec un numéro de colonne `awk` rend la commande indépendante du format d'horodatage du journal. Pour un seul fichier, `cat` est d'ailleurs superflu : `grep` lit directement le fichier.

bash

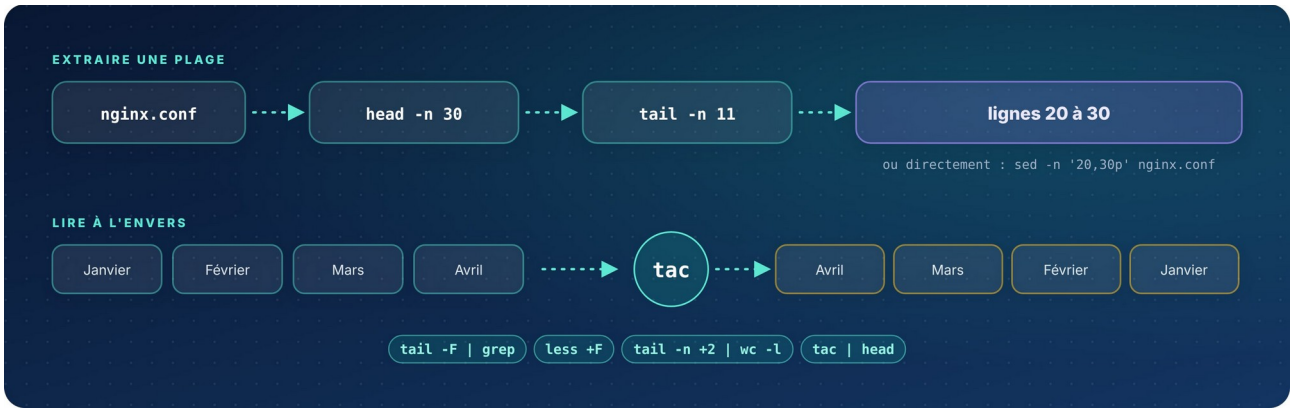
```
$ sudo grep 'Failed password' /var/log/auth.log | grep -oE 'from [0-9.]+' \
    | awk '{print $2}' | sort | uniq -c | sort -rn
# RHEL / Rocky / AlmaLinux : remplacer /var/log/auth.log par /var/log/secure
```

> « USELESS USE OF CAT »
> Le motif `cat fichier | commande` au lieu de `commande fichier` est surnommé « useless use of cat ». Il est sans danger mais crée un processus inutile. `cat` reste en revanche indispensable pour enchaîner **plusieurs** fichiers dans un même pipeline.




# Aller plus loin : combiner les commandes

La vraie puissance de ces outils apparaît lorsqu'on les combine entre eux et avec `sed`, `grep` ou `less`. Voici les combinaisons les plus utiles au quotidien.




## DANS CETTE PARTIE


| | | |
|---|---|---|
| 19 | **tac** | Lire un fichier à l'envers |
| 20 | **head \| tail** | Extraire une plage de lignes |
| 21 | **Recettes** | Les combinaisons du quotidien |

FICHE 19 · COMBINAISONS

# 19 tac — Lire un fichier à l'envers

| OBJECTIF | DISPONIBILITÉ | NIVEAU |
| --- | --- | --- |
| Voir les événements récents en premier | Toutes distributions (GNU coreutils) | Pratique |

`tac` — `cat` écrit à l'envers — affiche un fichier en commençant par sa **dernière** ligne. Sur un journal, les événements les plus récents apparaissent ainsi en premier.

bash

```
$ printf '%s\n' Janvier Février Mars Avril > mois.txt
$ tac mois.txt
Avril
Mars
Février
Janvier
# Les 20 événements les plus récents, du plus récent au plus ancien
$ sudo tac /var/log/syslog | head -n 20
```

Contrairement à `tail -f`, `tac` lit le fichier une seule fois puis s'arrête : c'est une vue ponctuelle, pas un suivi.

FICHE 20 · COMBINAISONS

# 20 head | tail — Extraire une plage de lignes

| OBJECTIF | DISPONIBILITÉ | NIVEAU |
|---|---|---|
| Afficher les lignes A à B d'un fichier | Toutes distributions (GNU coreutils) | Pratique |

Pour extraire une plage, on enchaîne `head` et `tail`. Lignes 20 à 30 d'un fichier : `head -n 30` garde les 30 premières lignes, puis `tail -n 11` en conserve les 11 dernières.




**Figure 5** — *Extraire les lignes 20 à 30 en deux étapes.*

```bash
$ head -n 30 /etc/nginx/nginx.conf | tail -n 11
        types_hash_max_size 2048;
        # server_tokens off;

        # server_names_hash_bucket_size 64;
        # server_name_in_redirect off;

        include /etc/nginx/mime.types;
        default_type application/octet-stream;

        ##
        # SSL Settings
```

Les lignes vides comptent : le résultat contient bien 11 lignes, même si certaines sont vides. Dans un script, `sed` ou `awk` sont plus directs :

```bash
$ sed -n '20,30p' /etc/nginx/nginx.conf
# Variante rapide sur un très gros fichier : s'arrêter après la ligne 30
$ sed -n '20,30p;30q' /etc/nginx/nginx.conf
$ awk 'NR>=20 && NR<=30' /etc/nginx/nginx.conf
```

**FORMULE GÉNÉRALE**

Lignes A à B : `head -n B fichier | tail -n (B − A + 1)`, ou plus simplement `sed -n 'A,Bp' fichier`.

FICHE 21 · COMBINAISONS

# 21 Recettes — Les combinaisons du quotidien

| OBJECTIF | DISPONIBILITÉ | NIVEAU |
| --- | --- | --- |
| Résoudre les besoins courants en une ligne | Toutes distributions (GNU coreutils) | Pratique |

| BESOIN | COMMANDE |
| --- | --- |
| Voir l'en-tête d'un CSV | `head -n 1 ventes.csv` |
| Afficher un CSV sans son en-tête | `tail -n +2 ventes.csv` |
| Vérifier la dernière ligne d'un fichier | `tail -n 1 fichier` |
| Suivre un journal en filtrant les erreurs | `sudo tail -F /var/log/syslog \| grep --line-buffered -i error` |
| Suivre un journal en pouvant remonter | `sudo less +F /var/log/syslog` |
| Lire un journal du plus récent au plus ancien | `sudo tac /var/log/syslog \| less` |
| Compter les lignes d'un fichier | `wc -l fichier` |
| Configuration sans commentaires ni lignes vides | `grep -Ev '^\s*(#\|$)' /etc/ssh/sshd_config` |
| Repérer des fins de ligne Windows | `cat -A fichier \| grep -c '\^M\$'` |
| Lignes 100 à 120 d'un gros fichier | `sed -n '100,120p;120q' fichier` |

**LESS +F**

`less +F` suit un fichier comme `tail -f` ; **Ctrl + C** suspend le suivi pour remonter et rechercher (`/motif`), puis **F** le relance. C'est le meilleur des deux mondes pour analyser un journal actif.

POUR CONCLURE

# Conclusion

`head`, `tail` et `cat` paraissent élémentaires, mais leurs options en font de véritables outils d'analyse. Bien maîtrisés, ils permettent d'inspecter une configuration, d'examiner le début ou la fin d'un journal, de suivre un service en direct ou d'assembler des fichiers — en quelques secondes, sans quitter le terminal.



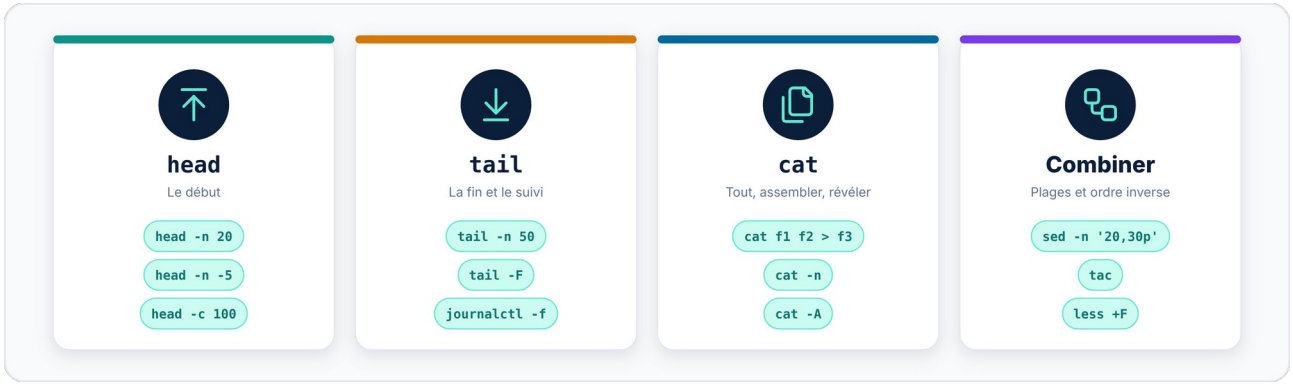




**Figure 6** — *Quel outil pour quel besoin : le résumé du guide.*

## Les réflexes à retenir

- `head` ou `tail` **avant** d'ouvrir un gros fichier dans un éditeur ;
- `tail -F`, et non `-f`, pour toute surveillance longue ;
- `journalctl -u <service> -f` lorsque le service écrit dans journald ;
- `cat -A` dès qu'un script ou une configuration se comporte de façon inexplicable ;
- `>>` pour ajouter, `>` pour remplacer — et `sudo tee` pour les fichiers système ;
- `sed -n 'A,Bp'` pour extraire une plage de lignes dans un script.

RESSOURCES

# Annexes

Ces annexes rassemblent les options à garder sous la main, l'emplacement des journaux selon la distribution, les pièges les plus fréquents et un glossaire.

## A. Aide-mémoire head

| OPTION | EFFET |
|---|---|
| `head -n N  (ou -N)` | Les N premières lignes (10 par défaut) |
| `head -n -N` | Tout sauf les N dernières lignes (GNU) |
| `head -c N` | Les N premiers octets (suffixes K, M, G acceptés) |
| `head -c -N` | Tout sauf les N derniers octets (GNU) |
| `head -q / -v` | Masquer / toujours afficher les en-têtes de fichiers |

## B. Aide-mémoire tail

| OPTION | EFFET |
|---|---|
| `tail -n N  (ou -N)` | Les N dernières lignes (10 par défaut) |
| `tail -n +N` | Depuis la ligne N jusqu'à la fin |
| `tail -c N` | Les N derniers octets |
| `tail -f` | Suivre le fichier ouvert (descripteur) |
| `tail -F` | Suivre par nom et réessayer (= --follow=name --retry) |
| `tail --pid=PID` | Arrêter le suivi à la fin du processus PID |
| `tail -s N` | Intervalle de vérification, en secondes |

## C. Aide-mémoire cat et tac

| OPTION | EFFET |
|---|---|
| `cat f1 f2` | Afficher les fichiers à la suite |
| `cat f1 f2 > f3` | Concaténer dans un nouveau fichier (remplace f3) |
| `cat f1 >> f2` | Ajouter f1 à la fin de f2 |
| `cat -n / -b` | Numéroter toutes les lignes / les lignes non vides |
| `cat -s` | Fusionner les lignes vides consécutives |
| `cat -A` | Révéler tabulations, fins de ligne, caractères spéciaux |
| `cat -E / -T` | Afficher les fins de ligne ($) / les tabulations (^I) |
| `tac fichier` | Afficher les lignes en ordre inverse |

## D. Où trouver les journaux ?

| CONTENU | DEBIAN / UBUNTU | RHEL / ROCKY / ALMA |
|---|---|---|
| Journal système général | `/var/log/syslog` | `/var/log/messages` |
| Authentification (SSH, sudo) | `/var/log/auth.log` | `/var/log/secure` |
| Noyau | `/var/log/kern.log` | `journalctl -k` |
| Tâches cron | `/var/log/syslog` | `/var/log/cron` |
| Gestionnaire de paquets | `/var/log/apt/history.log` | `/var/log/dnf.log` |
| Serveur web nginx | `/var/log/nginx/` | `/var/log/nginx/` |
| Services systemd | `journalctl -u <service>` | `journalctl -u <service>` |

**DEBIAN 12 ET CONTENEURS**

Depuis Debian 12, rsyslog n'est plus installé par défaut : `/var/log/syslog` peut donc être absent. Les journaux restent consultables avec `journalctl`.

## E. Pièges courants

| SYMPTÔME | CAUSE | SOLUTION |
|---|---|---|
| `tail -f` muet depuis la nuit | Le journal a été tourné par logrotate | Utiliser `tail -F` |
| `cat a b > a` a vidé `a` | Le shell tronque la destination avant la lecture | Écrire dans un fichier temporaire |
| `sudo cat >> fichier` refusé | La redirection est faite sans privilèges | `sudo tee -a fichier` |
| `bad interpreter: /bin/bash^M` | Fins de ligne Windows (CRLF) | `cat -A`, puis `sed -i 's/\r$//'` ou `dos2unix` |
| Terminal illisible après un `cat` | Un fichier binaire a été affiché | Taper `reset` |
| `head -n -5` refusé | Version BSD/macOS de head | GNU coreutils (`ghead`) |
| `/var/log/syslog` introuvable | rsyslog absent (Debian 12, conteneurs) | `journalctl` |

## F. Sources et références


- Anusha Saive, « Afficher et gérer des fichiers à l'aide des commandes head, tail et cat sous Linux », Tecmint (tecmint.com), mis à jour le 27 août 2026. Article dont ce guide reprend le plan et qu'il enrichit.
- Pages de manuel Linux : `head(1)`, `tail(1)`, `cat(1)`, `tac(1)`, `journalctl(1)`, `sed(1)`, `less(1)`.
- GNU Coreutils — manuel officiel (gnu.org/software/coreutils/manual).

## G. Glossaire

| TERME | DÉFINITION |
|---|---|
| Sortie standard (stdout) | Flux où une commande écrit son résultat ; par défaut, le terminal |
| Entrée standard (stdin) | Flux d'où une commande lit ses données ; par défaut, le clavier |
| Redirection | `>` envoie la sortie dans un fichier (en le remplaçant), `>>` l'ajoute à la fin |
| Tube (pipe) | `\|` relie la sortie d'une commande à l'entrée de la suivante |
| Descripteur de fichier | Référence interne qu'un processus garde vers un fichier ouvert |
| Rotation des journaux | Archivage périodique d'un journal (renommage, compression) par logrotate |
| Heredoc | Bloc de texte fourni à une commande directement dans la ligne de commande (`<< EOF`) |
| EOF · Ctrl + D | Fin de fichier : signale à une commande que la saisie est terminée |
| LF · CRLF | Fin de ligne Unix (`\n`) et Windows (`\r\n`) ; le `\r` s'affiche `^M` |
| Octet | Unité de 8 bits ; un caractère accentué en occupe plusieurs en UTF-8 |
| GNU coreutils | Ensemble des utilitaires de base de Linux, dont head, tail, cat et tac |
| journald | Service de journalisation de systemd, consulté avec journalctl |

DOCUMENT RÉDIGÉ ET MIS EN FORME PAR

**Ngouye GNING**

Version 1.0 · Octobre 2026